

SCD 2 cu triggere

SCD Type 2 (Slowly Changing Dimension Type 2) este o tehnică folosită în data warehousing pentru a gestiona **modificările istorice ale datelor din dimensiuni**.

Tabelele de dimensiuni (*Dimension Tables*) sunt componente esențiale într-un **data warehouse**, folosite pentru a **descrie** datele din **tabelele de fapt (*Fact Tables*)**. Ele conțin **informații descriptive** sau categorice.

Exemplu:

Dacă ai un **tabel de fapte** vanzari cu:

- id_client
- id_produș
- cantitate
- suma

Atunci ai nevoie de:

- dim_clienti: cu detalii despre clienți (nume, adresă etc.)
- dim_produșe: cu detalii despre produse (nume, categorie, preț etc.)

Scop: oferă **conținut semantic** analizei.

Unele attribute ale entităților **se schimbă rar în timp**, dar trebuie **urmărite istoric**. De exemplu:

- Clientul își schimbă adresa.
- Produsul este rebranduit.
- Un angajat trece la alt departament.

Atunci, **SCD 2 păstrează versiunea veche și adaugă una nouă** (NU se **suprascriu** datele, ci se păstrează **istoricul** complet al modificărilor).

Type 2 înseamnă că la fiecare modificare importantă:

- ✓ Se **închide** versiunea curentă (cu valid_to = sysdate).
- ✓ Se **crează o nouă versiune** a rândului (cu valid_from = sysdate).
- ✓ Se marchează care rând este „actual” (ex: is_current = 'Y').

Este o metodă **istorică** și **non-distructivă**, ideală pentru:

- ✓ **Audit** (cine, când, ce a schimbat).
- ✓ **Raportare istorică** (ce știam despre client în 2022?).
- ✓ **Analize longitudinale**.

Avantajele implementării SCD Type 2 cu trigger

1. Automatizare și centralizare a logicii

- Triggerul gestionează automat închiderea versiunii vechi și inserarea celei noi, fără a fi nevoie ca aplicațiile sau procedurile externe să conțină această logică.
- Asigură consistență la nivelul bazei de date.

2. Transparență pentru aplicație

- Aplicația poate face un simplu UPDATE, iar triggerul are grijă să creeze noua versiune și să o închidă pe cea veche.
- Scade riscul de erori umane.

3. Ușor de întreținut în cazuri simple

- Pentru tabele de dimensiuni cu structură clară și actualizări ocazionale, triggerul e simplu de întreținut și testat.

4. Funcționează în timp real

- Nu este nevoie de joburi batch sau procese ETL pentru a aplica modificările: modificarea este capturată imediat la momentul UPDATE-ului.

Limitările implementării SCD Type 2 cu trigger

1. Performanță

- Triggerul se execută pentru fiecare rând modificat, ceea ce poate afecta performanța în cazuri cu volume mari de date (ex: bulk update).
- Ineficient pentru operațiuni MERGE sau BULK COLLECT/FORALL, unde triggerul este apelat individual pe fiecare rând.

2. Complexitate crescută

- Debugging-ul poate deveni dificil (nu vezi direct ce se întâmplă dacă logica e „ascunsă” în trigger).
- Poate produce efecte neașteptate dacă nu este bine testat (ex: modificări accidentale generează rânduri noi nedorite).

3. Dificultăți la controlul tranzacțiilor

- Dacă o aplicație dorește să controleze în mod explicit istoricul (de ex., să insereze manual o versiune), triggerul poate încurca logica.
- Triggerul trebuie să fie atent la duplicate, la ordinea operațiunilor și la datele deja existente.

4. Limitări funcționale

- Triggerul nu poate gestiona ușor situații mai complexe:
- Detectarea modificărilor doar pe anumite coloane.
- Calcul de versiuni bazate pe reguli de business mai complexe.
- Crearea de loguri auxiliare, notificări sau integrare cu alte sisteme.

5. Mai greu de extins

- Dacă trebuie adăugate validări suplimentare sau să gestionezi mai multe tipuri de modificări, triggerul poate deveni greu de întreținut.

Exemplu de SCD Type 2 cu trigger

```
-- =====
-- CURĂȚARE (OPTIONAL)
-- =====

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE dim_employees CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE src_employees CASCADE CONSTRAINTS';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

-- =====
-- TABEL SURSĂ: SRC_EMPLOYEES
-- =====

CREATE TABLE src_employees (
    emp_id    NUMBER PRIMARY KEY,
    full_name  VARCHAR2(100),
    department VARCHAR2(100),
    salary     NUMBER,
    last_updated DATE
);

-- =====
-- TABEL ISTORIC: DIM_EMPLOYEES (istoricul complet al clienților)
-- =====

CREATE TABLE dim_employees (
    surrogate_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY
KEY, --cheia primară internă (identifică fiecare versiune)
    emp_id    NUMBER,
    full_name  VARCHAR2(100),
    department VARCHAR2(100),
    salary     NUMBER,
    start_date DATE, -- când a fost adăugată versiunea
    end_date   DATE, -- când a fost înlocuită (sau NULL dacă e actuală)
    current_flag CHAR(1) CHECK (current_flag IN ('Y', 'N')) -- 'Y' pentru activă, 'N' pentru veche
);
```

```

-- =====
-- INSERĂRI INIȚIALE ÎN SRC_EMPLOYEES
-- =====

BEGIN
    INSERT INTO src_employees VALUES (201, 'Victor Marinescu', 'FINANCE', 7100,
SYSDATE);
    INSERT INTO src_employees VALUES (202, 'Ioana Dragomir', 'HR', 4800, SYSDATE);
    INSERT INTO src_employees VALUES (203, 'Mihai Constantinescu', 'ENGINEERING', 8200,
SYSDATE);
    INSERT INTO src_employees VALUES (204, 'Sorina Popescu', 'LOGISTICS', 5300,
SYSDATE);
    COMMIT;
END;
/

-- =====
-- ÎNCĂRCARE VERSIUNI CURENTE ÎN DIM_EMPLOYEES
-- =====

BEGIN
    INSERT INTO dim_employees (
        emp_id, full_name, department, salary,
        start_date, end_date, current_flag
    )
    SELECT
        emp_id, full_name, department, salary,
        SYSDATE, NULL, 'Y'
    FROM src_employees;
    COMMIT;
END;
/

-- =====
-- TRIGGER SCD2 PE SRC_EMPLOYEES
-- =====

CREATE OR REPLACE TRIGGER trg_scd2_employees
BEFORE UPDATE ON src_employees
FOR EACH ROW
DECLARE
    v_changed BOOLEAN := FALSE;
BEGIN
    -- Detectăm schimbări pe coloanele urmărite

```

```

IF :NEW.department != :OLD.department
OR :NEW.salary    != :OLD.salary
OR :NEW.full_name != :OLD.full_name THEN
    v_changed := TRUE;
END IF;

IF v_changed THEN
    -- 1. Expirăm versiunea veche
    UPDATE dim_employees
    SET end_date = SYSDATE,
        current_flag = 'N'
    WHERE emp_id = :OLD.emp_id
        AND current_flag = 'Y';

    -- 2. Inserăm versiunea nouă
    INSERT INTO dim_employees (
        emp_id, full_name, department, salary,
        start_date, end_date, current_flag
    ) VALUES (
        :NEW.emp_id, :NEW.full_name, :NEW.department, :NEW.salary,
        SYSDATE, NULL, 'Y'
    );
END IF;
END;
/

-- =====
-- TESTARE
-- =====

-- Schimbăm salariul lui Victor
BEGIN
    UPDATE src_employees
    SET salary = 7600
    WHERE emp_id = 201;
    COMMIT;
END;
/

-- Schimbăm departamentul Sorinei
BEGIN

```

```
UPDATE src_employees  
SET department = 'HR'  
WHERE emp_id = 204;  
COMMIT;  
END;  
/
```

```
-- Verificăm istoricul complet pentru Victor și Sorina  
SELECT * FROM dim_employees  
WHERE emp_id IN (201, 204)  
ORDER BY emp_id, start_date;
```

Ce învățăm din acest exemplu

- Nu suprascriem datele, ci **le păstrăm în versiuni istorice**.
- Folosim **triggere** pentru a face asta automat.
- Evităm pierderea accidentală a datelor (ex: ștergeri definitive).

Link-uri utile:

- [Understanding Slowly Changing Dimensions](#)
- [Using Slowly Changing Dimensions](#)
- [Slowly changing dimension type 2 - Microsoft Fabric | Microsoft Learn](#)